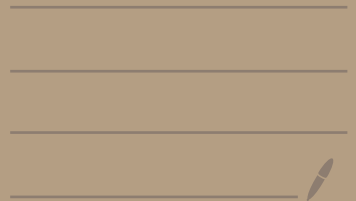


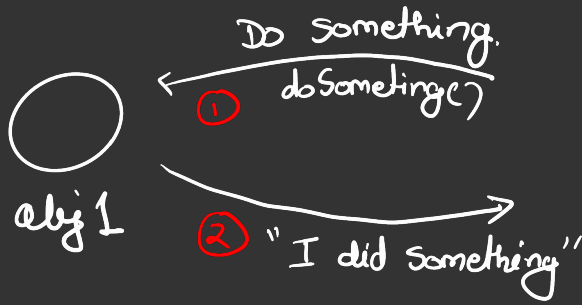
Decorator Pattern



(*) Decorator Pattern

Aim :-

Let's say mere pass ek object hai obj 1 , Mujhe is object ko additional responsibility provide karni hai runtime pe .



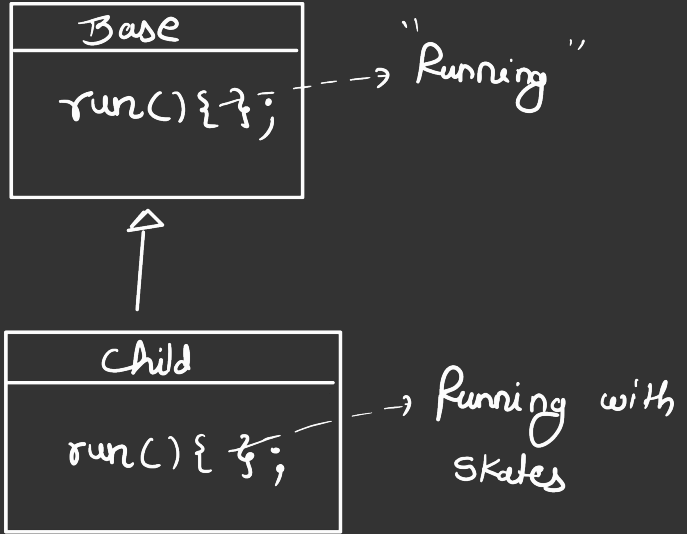
What I want now is, ye jo response aata hai "I did something" isko mai dynamically change kar saku .

For eg :- Agar mai isko firse doSomething() bole vo bole → "I did something great"

⌘ Now we do something same in Inheritance 😊
why we need this design Pattern?

→

Inheritance



Base b = new Child
b.run()

→ yaha pe
ja likhe ho
uske hisab
se o/p decide
hoga.

Reason 1 $\Rightarrow$ Inheritance is Bad [1st Design Pattern]

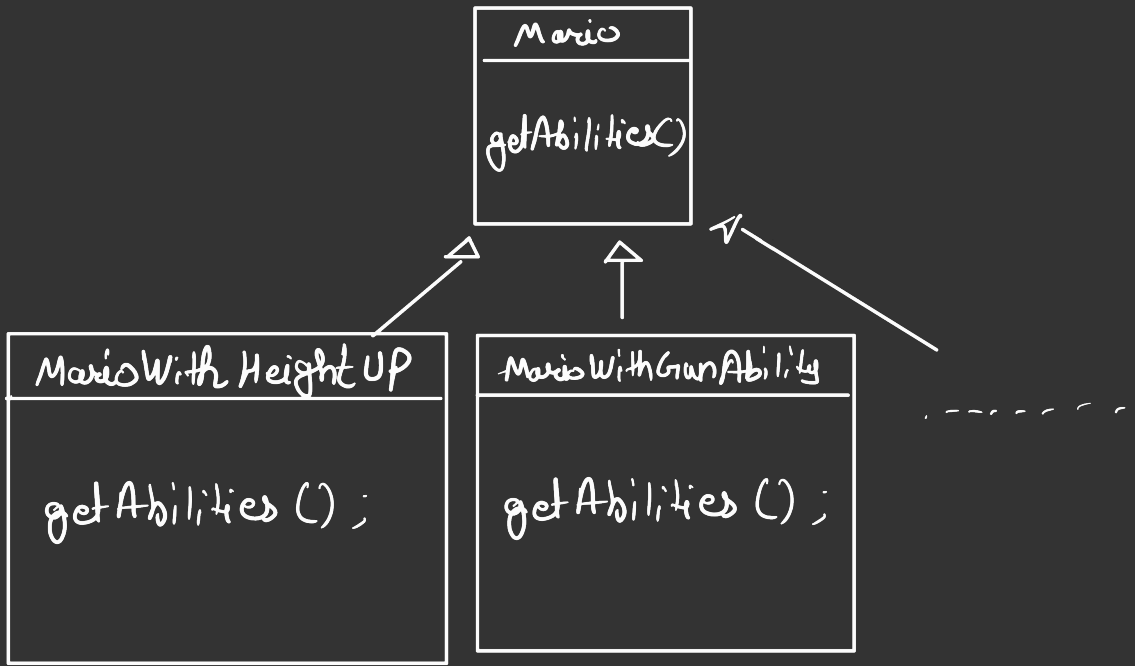
- $\hookrightarrow$ Multiple Hierarchy
 - $\hookrightarrow$ class Explosion.
-

eg: $\rightarrow$ Mario Game



I want with time, Mario Ki functionality enhance hojaye, ie additional responsibility add hojaye is response ke pass.

⊛ Agar hum ise inheritance se karke to kya hata?



lots of classes
☹️

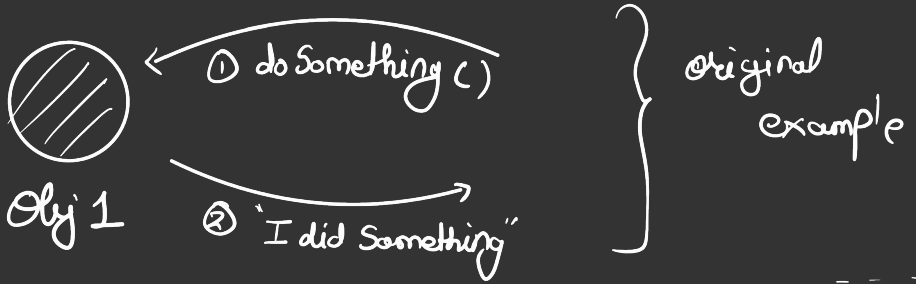
Class Explosion ⇒ Humare pass itni sari child class ban jaye, aur har change ke sath ek nayi child class humari ban rahi ho.



very complex to manage

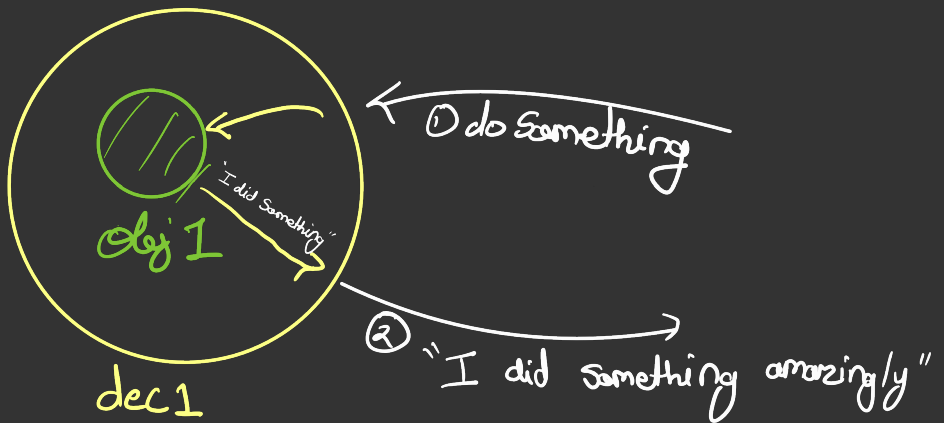
We should prefer Composition over Inheritance

* Decorator Pattern



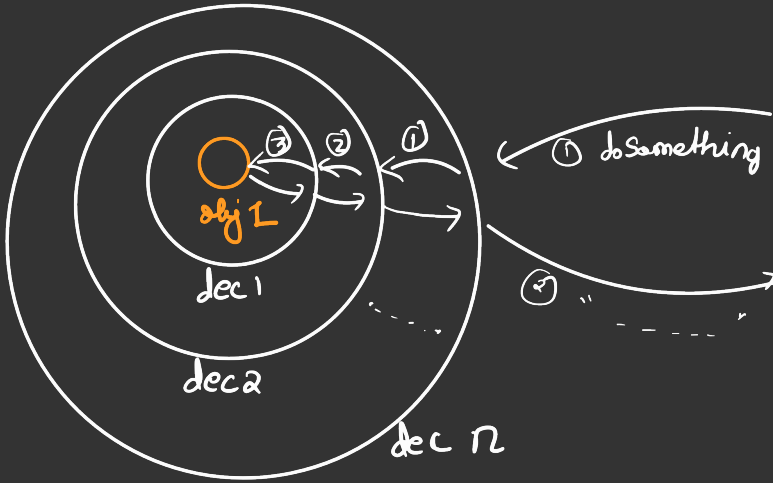
I want different obj for same `doSomething()` method dynamically at runtime.

→ Decorate the object (Obj 1)



↳ We can infinitely stack up decorators.

eg:-)



To request mai, obj 1 ko bhej raha hu wahi request mai decorator ko bhej raha hu, iska matlab decorator bhi ek object hai aur relationship hai

"IS-A" [Inheritance abhi bhi use hoga hai]

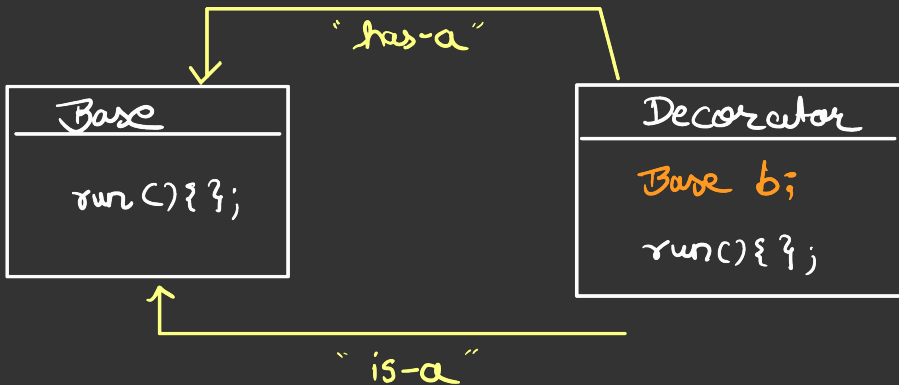
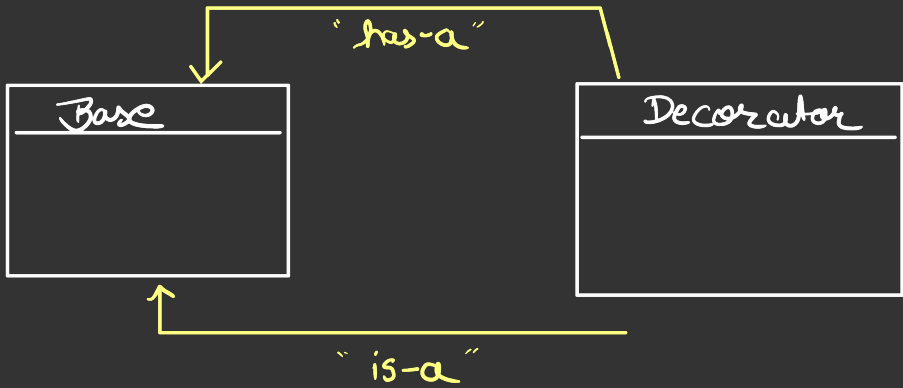
① Decorator should behave as an object so it is-a object.

② Decorator also has-a object [Composition]

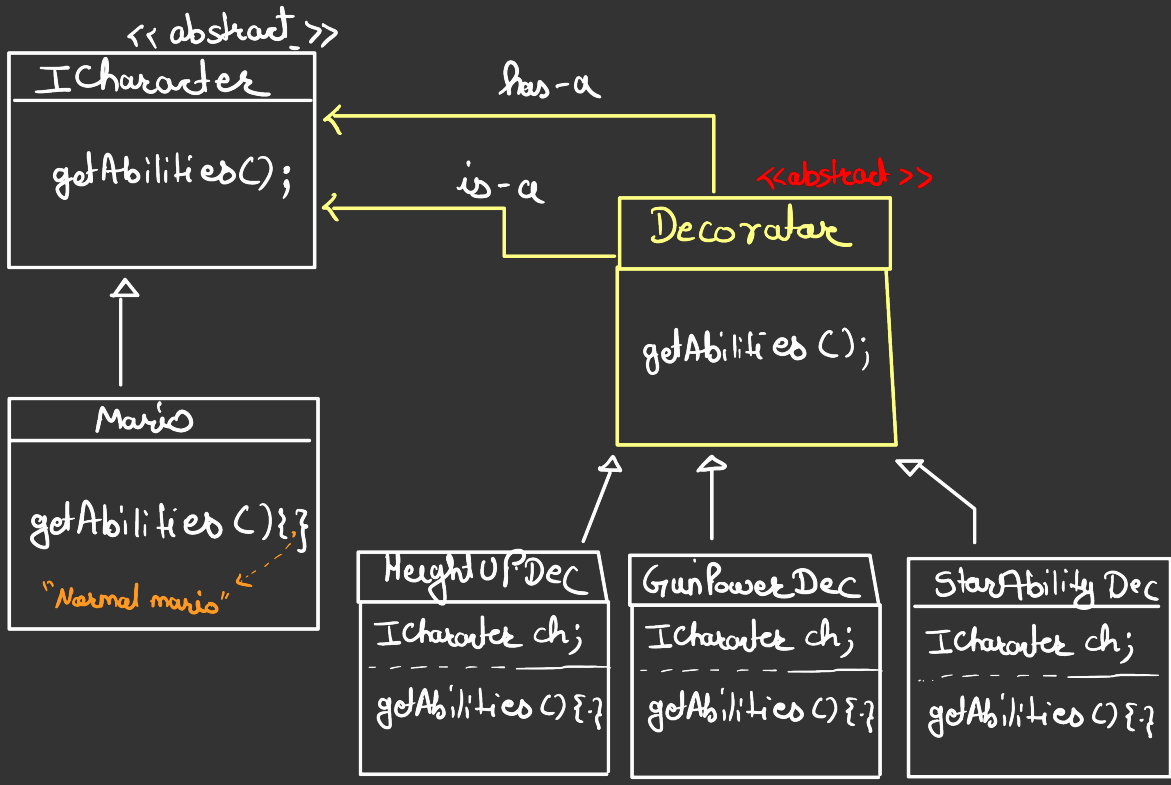
① Decorator is using **IS-A** for behaving as an object [not for dynamically changing behaviour]

② Decorator is using **HAS-A** for dynamically changing behaviour/properties.

③ UML



Mario Use Case



Now, this is interesting because mai in-decorators ko kisi bhi tarike se , kitne bhi jod sakta hu.

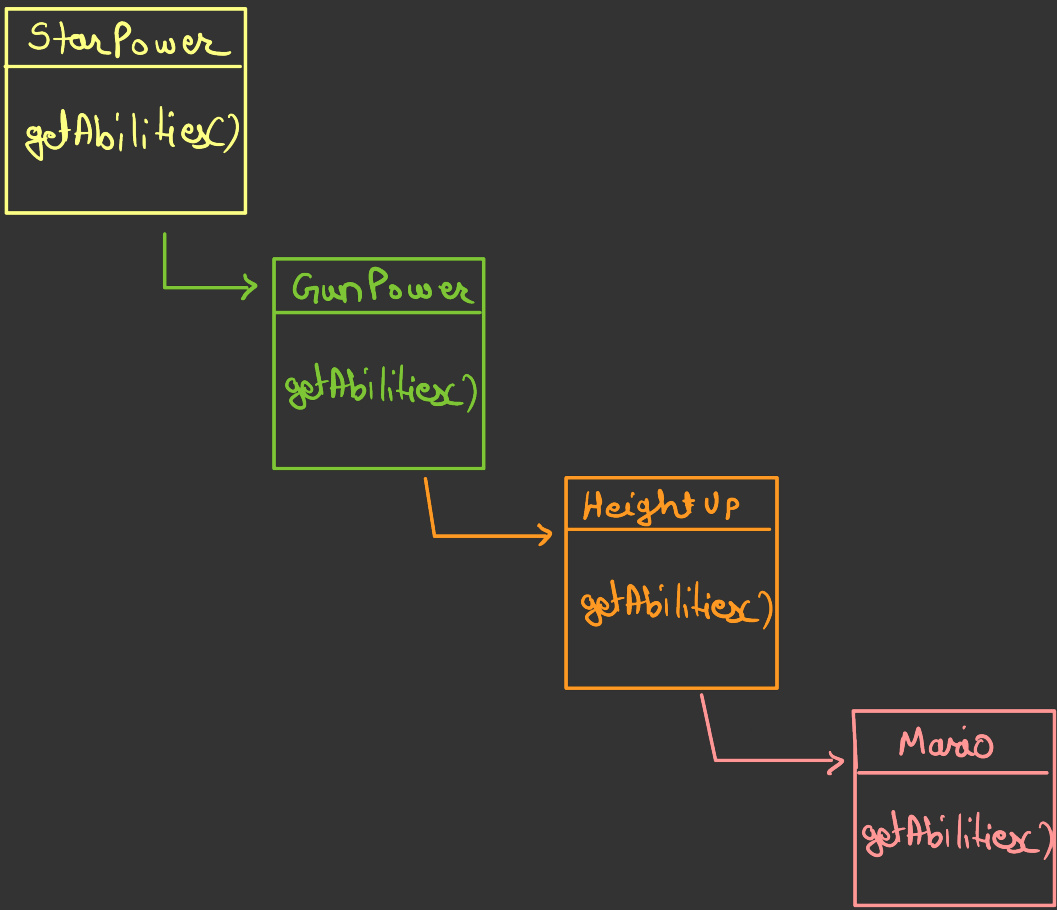
eg :-

```

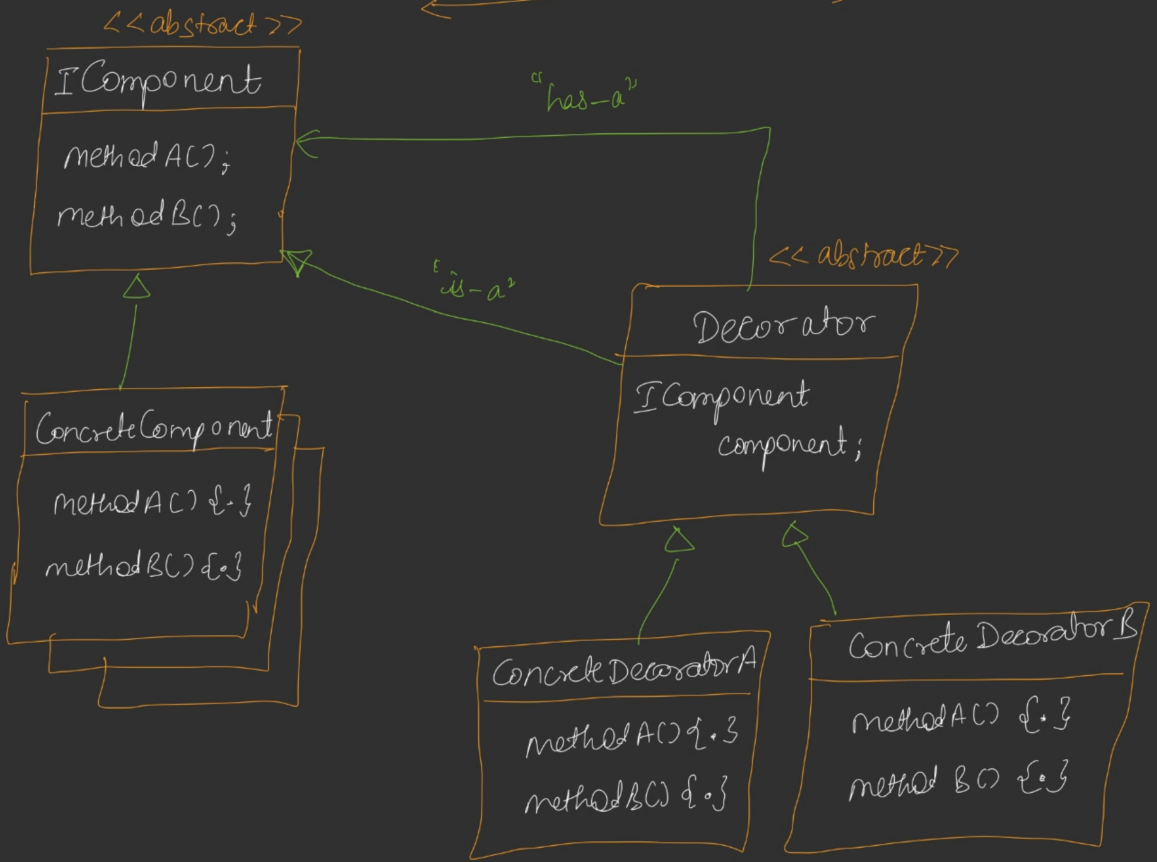
ICharacter mario = new HeightUPDecorator(
    new GunPowDec (
        new Mario() );
    )
)
    
```

①

```
Icharacter mario = new StarPower(new GunPower(  
    new HeightUP(  
        new Mario()) )) );
```



Standard UML

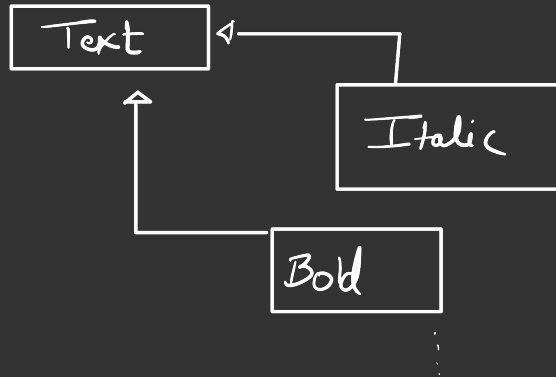


Standard Definition

➔ Decorator pattern attaches additional responsibilities to an object dynamically. Decorator provides a flexible alternative to subclassing for extending functionality.

* Real World Examples

→ ① Text Editor.



② Form Validator

① email validation Decorator.

② Phone-no validation Decorator

③ SQL-Injection validation Decorator

⋮

